Next Steps — Databricks Fundamentals

Next Steps After Databricks Fundamentals

> **Databricks Fundamentals** gives you the foundation.
> This guide shows you where to go next — what to learn, how to certify, and what to build.

Learning Path Overview

```
Databricks Fundamentals (today)
        ↓
Databricks Explorer (intermediate)
        ↓
Databricks Associate / Professional (advanced)
        ↓
Specialty Tracks: ML, Data Engineering, SQL Analytics
```

---

1. Certifications

✅ Databricks Certified Data Engineer Associate

- **What it covers:** Delta Lake, Spark, Databricks platform, Medallion Architecture, basic ETL
- **Exam:** 45 questions, 90 minutes, multiple choice + hands-on lab
- **Who:** Anyone who completed this training + ~3 months hands-on practice
- **Prep:** academy.databricks.com/learn/learning-path
- **Cost:** ~$200 USD

✅ Databricks Certified Associate Developer for Apache Spark

- **What it covers:** PySpark (or Scala Spark) — DataFrames, SQL, transformations, actions
- **Exam:** 60 questions, 120 minutes
- **Who:** Developers who write a lot of PySpark code

- **Note:** Most useful if you work heavily with Spark outside of Databricks

📘 Databricks Certified Data Engineer Professional

- **What it covers:** Advanced Delta Lake, DLT, Unity Catalog, performance, security
- **Prerequisites:** Associate exam recommended
- **Who:** Senior data engineers, architects

---

2. AutoLoader — Production File Ingestion

AutoLoader (`cloudFiles`) is the recommended way to ingest from cloud storage incrementally at scale. It replaces manual `spark.read` + full scans.

Why use it?

- Automatically detects **new files** arriving in a directory (no full re-scan)
- Native support for S3, ADLS Gen2, GCS
- Built-in **schema inference and evolution**
- Works seamlessly with **Lakeflow Pipelines** (the declarative pipeline standard)

Quick Pattern

```python
# Streaming read from cloud storage — AutoLoader
df = (
    spark.readStream
        .format("cloudFiles")
        .option("cloudFiles.format", "json")          # or csv, parquet, avro, etc.
        .option("cloudFiles.schemaLocation", "/path/to/schema_checkpoint")
        .load("/mnt/landing/orders/")
)

# Write to Bronze Delta table — micro-batches
(
    df.writeStream
        .format("delta")
        .option("checkpointLocation", "/path/to/checkpoint")
        .outputMode("append")
        .trigger(availableNow=True)                   # process all available, then
        stop
        .toTable("catalog.bronze.orders")
)
```

Key Options

Option	Description
`cloudFiles.format`	Source format: `json`, `csv`, `parquet`, `avro`
`cloudFiles.schemaLocation`	Where to persist inferred schema (DBFS or Unity Volume)
`cloudFiles.inferColumnTypes`	Auto-infer types from JSON/CSV (default: all strings)
`cloudFiles.maxFilesPerTrigger`	Batch size control

📖 AutoLoader documentation

---

3. Lakeflow Pipelines

Lakeflow Pipelines is the current product name for what was previously called Delta Live Tables (DLT). It is Databricks' declarative framework for building **reliable, maintainable, production ETL pipelines**. You define tables with Python or SQL; Databricks handles orchestration, incremental processing, data quality, and restarts automatically.

> **Note:** The `dlt` Python library and decorator API (`@dlt.table`, `@dlt.expect_or_drop`, etc.) remain unchanged — only the product branding changed. Code written for DLT works as-is on Lakeflow Pipelines.

Key Concepts

```python
import dlt
from pyspark.sql.functions import col, sum as _sum

# Bronze — raw ingestion via AutoLoader (cloud files)
@dlt.table(
    comment="Raw orders from daily file drop",
    table_properties={"quality": "bronze"}
)
def orders_bronze():
    return (
        spark.readStream
            .format("cloudFiles")
            .option("cloudFiles.format", "json")
            .option("cloudFiles.schemaLocation", "/pipeline/schema/orders")
            .load("/landing/orders/")
    )

# Silver — cleaned & validated (data quality rules enforced at write)
@dlt.table(
    comment="Validated orders — no nulls, no negatives",
    table_properties={"quality": "silver"}
)
@dlt.expect_or_drop("valid_order_id",  "order_id IS NOT NULL")
@dlt.expect_or_drop("positive_amount",  "total_amount > 0")
@dlt.expect("known_status",            "status IN
         ('PENDING','COMPLETED','CANCELLED')")
def orders_silver():
    return (
        dlt.read_stream("orders_bronze")
            .withColumn("order_date", col("order_datetime").cast("date"))
            .drop("order_datetime")
    )

# Gold — aggregated (batch read, not streaming)
@dlt.table(
    comment="Daily revenue by payment method",
    table_properties={"quality": "gold"}
)
def revenue_by_day():
    return (
        dlt.read("orders_silver")
            .groupBy("order_date", "payment_method")
            .agg(_sum("total_amount").alias("daily_revenue"))
    )
```

Why Lakeflow Pipelines over manual notebooks?

- Built-in **data quality** rules (`expect`, `expect_or_drop`, `expect_or_fail`) with quality metrics dashboard

- Automatic **dependency resolution** — Databricks infers the DAG from `dlt.read()` calls
- **Incremental processing** by default — only new/changed data is processed per run
- **Serverless mode** available — no cluster management needed
- Pipeline observability: flow graph, quality metrics, event log, all in the UI
- Supports both Python and SQL in the same pipeline
- Enhanced error handling: failed quality checks quarantined, not lost

Lakeflow vs. manual notebooks

	Manual Notebooks	Lakeflow Pipelines
Dependency order	Manual, error-prone	Automatic DAG
Data quality	`assert` or custom code	`@dlt.expect_*` built-in
Incremental load	Write from scratch	Built-in, default
Recovery on failure	Manual rerun whole job	Rerun from failed table
Observability	`print()` / logging	Visual pipeline graph

📖 Lakeflow Pipelines docs

---

4. Unity Catalog — Advanced Features

You used Unity Catalog today for basic `CATALOG.SCHEMA.TABLE` addressing. The platform goes much further:

Row-Level Security

```
-- Only show rows matching the user's region
CREATE ROW ACCESS POLICY region_policy
  AS (region STRING) RETURNS BOOLEAN
  USING (region = current_user_region());

ALTER TABLE sales.transactions
  SET ROW ACCESS POLICY region_policy ON (region);
```

Column Masking

```sql
-- Mask email for non-admin users
CREATE COLUMN MASK email_mask
  AS (email STRING) RETURNS STRING
  USING (
    IF(is_member('data_admins'), email, REGEXP_REPLACE(email, '(.+)@', '***@'))
  );

ALTER TABLE customers ALTER COLUMN email SET MASK email_mask;
```

Data Lineage

- Automatic visual lineage graph: see which tables a column came from
- Available in **Catalog Explorer** → **Table** → **Lineage** tab
- No configuration needed — Unity Catalog tracks it automatically

Tags & Classifications

```sql
-- Tag a table for governance
ALTER TABLE customers SET TAGS ('pii' = 'true', 'domain' = 'customer');

-- Tag a column
ALTER TABLE customers ALTER COLUMN email SET TAGS ('sensitivity' = 'high');
```

📖 Unity Catalog docs

---

5. Structured Streaming

AutoLoader is built on Structured Streaming. Understanding the core concepts unlocks real-time pipelines:

```python
# Read from Event Hub / Kafka
from pyspark.sql.functions import from_json, col

schema = StructType([...])

stream_df = (
    spark.readStream
        .format("kafka")
        .option("kafka.bootstrap.servers", "your-kafka:9092")
        .option("subscribe", "orders-topic")
        .load()
        .select(from_json(col("value").cast("string"), schema).alias("data"))
        .select("data.*")
)

# Windowed aggregation — 5-minute tumbling window
from pyspark.sql.functions import window

windowed = (
    stream_df
        .groupBy(window("event_time", "5 minutes"), "payment_method")
        .agg(F.sum("total_amount").alias("revenue_5min"))
)

# Write to Delta with checkpointing
(
    windowed.writeStream
        .format("delta")
        .outputMode("complete")          # or "append" for append-only
        .option("checkpointLocation", "/chk/revenue_5min")
        .toTable("catalog.gold.revenue_realtime")
)
```

Trigger Types

Trigger	When to use
`trigger(processingTime="1 minute")`	Micro-batch, low latency
`trigger(availableNow=True)`	Process backlog then stop (like batch)
`trigger(once=True)`	Deprecated — use `availableNow` instead
Continuous	Sub-second latency (experimental)

📖 Structured Streaming guide

6. Performance Optimization

OPTIMIZE + ZORDER

```
-- Compact small files (run daily / weekly)
OPTIMIZE catalog.silver.customers;

-- ZORDER: co-locate related data on disk for fast filtered reads
OPTIMIZE catalog.silver.orders ZORDER BY (customer_id, order_date);
```

When to use Z-ORDER: - Columns you filter on frequently in `WHERE` clauses - Columns used in `JOIN` conditions - High-cardinality columns (customer_id, product_id, date)

Liquid Clustering (Databricks 13.3+)

```
-- Better than ZORDER — incremental, no full rewrite needed
CREATE TABLE orders
  CLUSTER BY (customer_id, order_date)
  AS SELECT * FROM raw_orders;

-- Re-cluster incrementally (not full rewrite)
CLUSTER TABLE orders;
```

Photon Engine

- Enabled automatically on **Databricks Runtime 9.1+** with compatible clusters
- Re-implemented vectorized Spark execution in C++
- 2–10x faster for SQL-heavy workloads, GROUP BY, JOINS
- No code changes required

Adaptive Query Execution (AQE)

- Enabled by default (Spark 3.0+)
- Dynamically changes join strategies, merges small partitions
- Can disable per-query if needed: `spark.conf.set("spark.sql.adaptive.enabled", "false")`

7. MLflow — Experiment Tracking

```python
import mlflow
import mlflow.sklearn
from sklearn.ensemble import RandomForestClassifier

with mlflow.start_run(run_name="rf_experiment_v1"):
    model = RandomForestClassifier(n_estimators=100, max_depth=5)
    model.fit(X_train, y_train)

    # Log parameters
    mlflow.log_param("n_estimators", 100)
    mlflow.log_param("max_depth", 5)

    # Log metrics
    mlflow.log_metric("accuracy", model.score(X_test, y_test))

    # Log the model itself
    mlflow.sklearn.log_model(model, "model")
```

- MLflow is **built into every Databricks workspace** — no installation needed
- Access via **Experiments** tab in the left sidebar
- Supports: sklearn, XGBoost, PyTorch, TensorFlow, Hugging Face, LangChain

📖 MLflow on Databricks

---

8. Databricks Workflows — Advanced Orchestration

Beyond single-notebook jobs, Workflows support:

```python
# In any notebook: return structured output to the parent job
dbutils.notebook.exit(json.dumps({"rows_processed": 15_000, "status": "ok"}))

# Parent orchestrator reads it:
result = dbutils.notebook.run("./child_notebook", timeout_seconds=600,
                              arguments={"date": "2025-01-15"})
import json
data = json.loads(result)
print(data["rows_processed"])  # 15000
```

Multi-task Job Features

- **Task dependencies** — DAG of tasks with `depends_on`
- **Conditional branching** — `if/else` task logic based on previous task output

- **Repair & rerun** — rerun only failed tasks, not the full pipeline
- **Parameterized jobs** — pass different dates, environments at runtime
- **Webhooks & alerts** — notify Slack/Teams/PagerDuty on failure

9. External Integrations

Tool	How to Connect
Power BI	Databricks connector → Partner Connect → pick Power BI
Azure Data Factory	ADF Linked Service → Databricks cluster + notebook activity
dbt	`dbt-databricks` adapter — install via `pip install dbt-databricks`
VS Code	Databricks extension for VS Code — sync notebooks, run cells locally
Tableau	ODBC/JDBC connector → Databricks SQL Warehouse endpoint
Great Expectations	`great_expectations` Python library — quality checks in notebooks

10. Community & Resources

Resource	Link
Databricks documentation	docs.databricks.com
Databricks Academy	academy.databricks.com
Community forums	community.databricks.com
Databricks Blog	databricks.com/blog
YouTube: Databricks	search “Databricks Data + AI Summit”
GitHub: Delta Lake	github.com/delta-io/delta

Generated for Databricks Fundamentals training · RetailHub scenario